PROJECT: SOLVING THE TRVELING SALESEMAN PROBLEM
USING META-HEURISTICS

1. Description

The travelling salesman problem (also called the traveling salesperson problem or TSP) asks the following question: "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?" It is an NP-hard problem in combinatorial optimization, important in theoretical computer science and operations research. The TSP problem was first formulated in 1930 and is one of the most intensively studied problems in optimization. It is used as a benchmark for many optimization methods.

The TSP has several applications even in its purest formulation, such as planning, logistics, and the manufacture of microchips. Slightly modified, it appears as a sub-problem in many areas, such as DNA sequencing. In these applications, the concept city represents, for example, customers, soldering points, or DNA fragments, and the concept distance represents travelling times or cost, or a similarity measure between DNA fragments.

Due to the NP-hardness of the problem, many meta-heuristic approaches such as Local Search, Simulated Annealing, Variable Neighborhood Search, Tabu Search, Genetic Algorithm, Ant Colony Optimization, Bee Algorithm, etc. have been applied to provide high-quality solutions in a reasonable amount of computational time. Some works have also combined more than one metaheuristic to solve the problem in the hope of benefiting from the high performance of every method.

This project aims at designing, implementing and assessing the performance of meta-heuristics to solve the TSP problem. More than one meta-heuristic can be applied and compared according to several metrics such as quality of obtained solutions, convergence time, stability, etc. New approaches can also be applied whereby several meta-heuristics can be combined together to solve the underlying problem.

The deliverables of this project come in the form of:

- Commented Source code showing the implementation of the selected meta-heuristics
- A report supporting the source code. It should include the detailed steps of the implemented methods, their workflows, design details, experimental setups, assessment analysis, as well as discussions and conclusions.
- A power point presentation summarizing the report and source code.

Two instances are provided to test the implemented methods: distances_between_cities_10.txt, distances_between_cities_50.txt. The optimal solutions for those instances are already known, for the first instance is 3473km and 5644km for the second. More well-known instances can be found here: http://www.math.uwaterloo.ca/tsp/data/index.html

Use Genetic Algorithm to solve TSP

1. Method and step introduction

Genetic algorithm is base on biological theory.

(Key word: genotype, phenotype, evolution, fitness, selection, reproduction, Crossover, mutation, coding, decoding, individual, population.)

We mainly use GA to solve TSP problem, first we have a population of feasible solution, then we use fitness function to pick out the parental population, then we use crossover and mutation technology to generate a child population, then we replace it with the original population.

Then repeat all the processes above.

One sentence to introduce: we constantly map a population to certain better population using a fitness degree.

Pseudo code:

```
Populaition P;

SolutionSet S;

newPopulation N = empty;

P=S;


string globalMin;


    while (!terminate())

    {

        for x ∈ p
```

```
        if x.fitness>standard
            N = N∩ x

    crossover(N)
    mutation(N)

    for each v ∈ N
        if(v.fitness>globalMin.fitness)
            globalMin=v

    P = N;
}
```

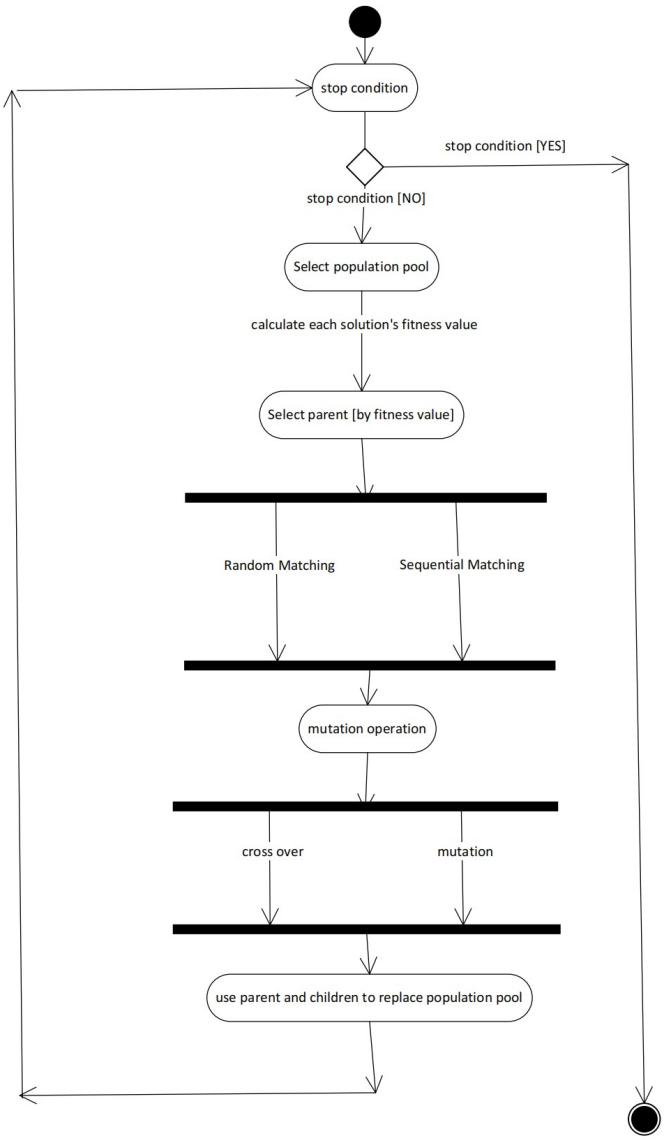
Now, specific steps:

1. Choose a population pool
2. Test stop condition (some solutions appears for constant times)
3. Evaluate the fitness value of each solution.
4. Rank the fitness value from high to low, if high, then the possibility of selection is also high.
5. Select highest 20 solutions as Elitism, and directly add those elites to the next generation, rest of the population we randomly select father and mother, then we reproduce offspring---crossover father and mother's chromosome, and generate child.
6. Mutate child's chromosome.
7. We select all the child and first 20 elites as the new generation.
8. Return to step2, is fulfilled, exit the program.

At last, in the process of cross over and mutation, the relationship between phenotype and genotype need to be build (e.g. phenotype is the city string, and genotype is your binary code after encoding).

2. Method flow chart

Activity diagram



3. Design details

In this algorithm, we mainly use 100 initial solutions to generate result, and get the average best result. Also we apply different neighborhood structure to

the Genetic Algorithm's mutation part, we will talk about it later. Final, we will derive the result about the GA' s quality of solution and solution's stability.

A. **Fitness Function:**

```
Fitness()
{
    For i=1 to pop.size
        For j=1 to chromosome.size-1
            Distance=Distance+cost<gene_j, gene_{j+1}>
        Pop[i].value=Distance
        Distance=0

    For i=1 to pop.size
        Pop[i].fitness=pop[i].#city/pop[i].value
}
```

You can see that, we use the $\frac{city\ \ number}{distance}$ as the fitness value.

B. Parent selection

A. Roulette Wheel

Solution x, y, z, first, use fitness function to get, f(x), f(y), f(z), then, use

B. Elitism

C. Stochastic Universal Sampling

In the select parent part, we select best 20 elite with highest fitness value. For the rest of the population, we will make them reproduce children by randomly mating, this means for every solution left, just mate with other solutions for random times. Then, we will collect all the offspring, and add all the elite to compose a new generation.

C. Encoding:

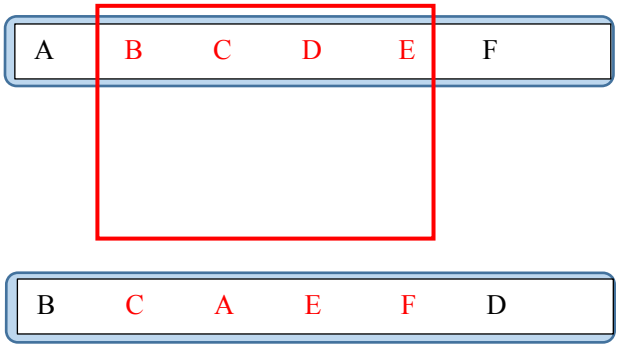
A. Binary coding

B. Floating point coding

C. Neither of them, we just use the city string as the code to do the cross over

D. Cross over:

only exchange the red part, but the red part's size and position can be random



E. Mutation.

Each allele on a chromosome (child) has a mutation factor, we call it <p>, after parents reproduce the child, we check each alleles, and generate a mutation factor <p1> sequentially, once <p1> greater than <p>, the allele will mutate and exchange with another random allele.

F. New Population Pool.

We use the 20 elite and the children to replace the original population pool.

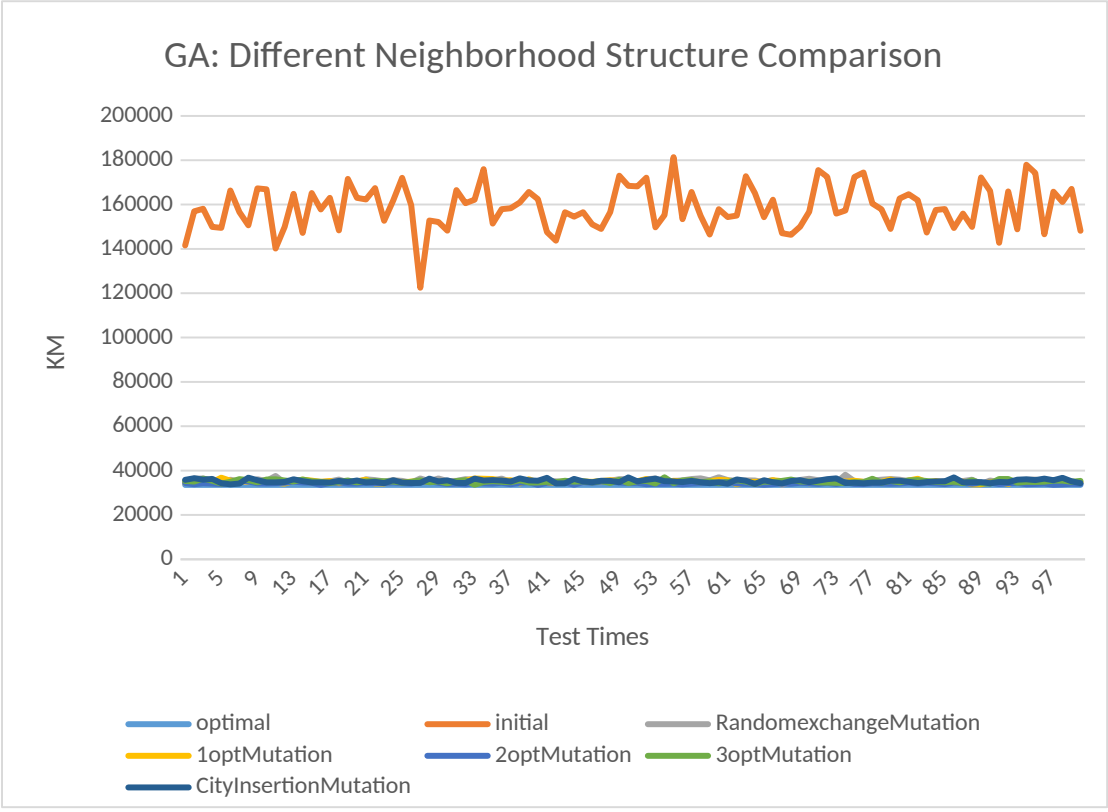
4. Experimental procedure

1. Development environment: Microsoft Visual Studio 2019
2. Operating system: Windows10
3. Configuration of the Computer:
   CPU: AMD Ryzen5 3500U 2.10GHz

RAM: 16GB

5. Assessment:

A. Quality of obtained solutions



(figure 1)

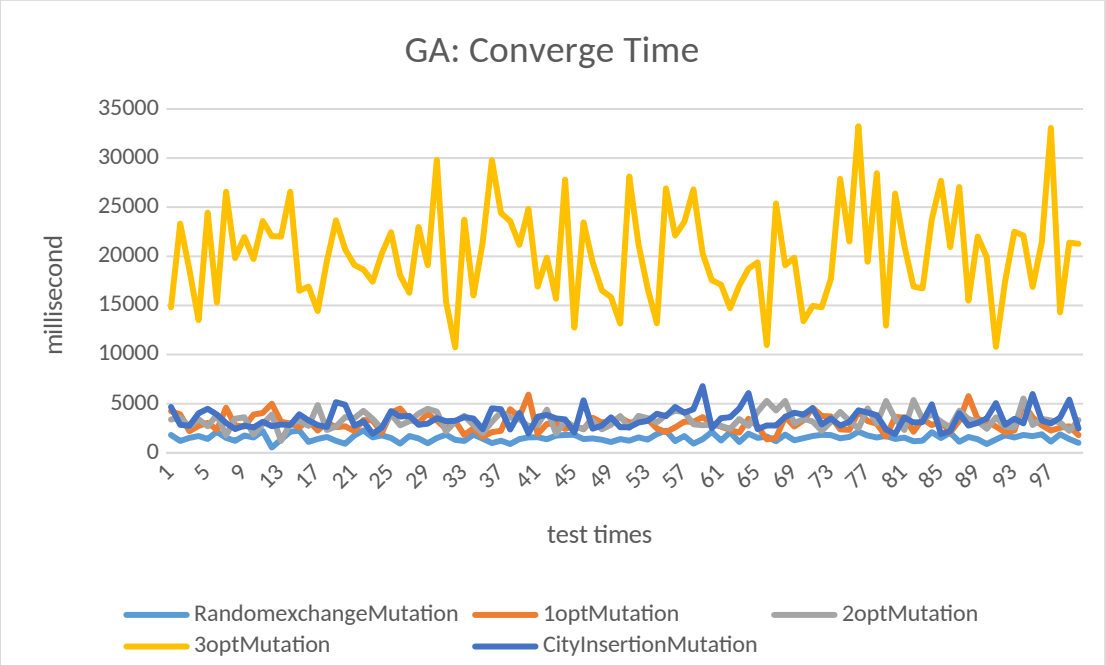
Solution							
	optimal	initial	RandomexchangeMutation	1optMutation	2optMutation	3optMutation	CityInsertionMutation
Average	33522	158540.4	35401.12	35055.32	34294.12	35083.45	35206.02
STDEVP	0	9805.093	715.8592918	598.5429288	379.5860977	578.1902347	755.7799809

(figure 2)

Figure 1 shows that, we implement GA on 100 initial solutions and get their final optimal solutions, however, we think that, different mutation structure may influence the mutation, so we use 5 different neighborhood structures to conduct the mutation. If you remember, our original mutation is to swap the allele with a random allele, now, we just use neighborhood structure on the entire city string. For example, A-B-D-F-E-G-H, after implement 3-OPT, we let i=E, j=A, we choose B, G to swap, we get A-G-D-F-E-B-H.

From the figure 2, we can say that, for the quality of the solution of GA, 2OPT>1OPT>3OPT>City Insertion>Random exchange Mutation (> means quality is better). from the figure 1, we can also tell that the all the 5 neighborhood structures applied in mutation can bring us the optimal solution.

B. Convergence time



(figure 3)

Time					
	RandomexchangeMutation	1optMutation	2optMutation	3optMutation	CityInsertionMutation
Average	1530.53	3047.65	3300	20227.93	3461.02
STDEVP	354.7907962	859.6008187	826.1069906	4849.569014	947.4829601

(figure 4)

From figure 3 and 4, it shows that converge time of GA applying 5 neighborhood structures has the following order:

Random exchange<1OPT<2OPT<City Insertion<3OPT, it means that GA with random exchange mutation will converge in the fastest way.

C. Stability

Solution							
	optimal	initial	RandomexchangeMutation	1optMutation	2optMutation	3optMutation	CityInsertionMutation
Average	33522	158540.4	35401.12	35055.32	34294.12	35083.45	35206.02
STDEVP	0	9805.093	715.8592918	598.5429288	379.5860977	578.1902347	755.7799809

(figure 5)

Time					
	RandomexchangeMutation	1optMutation	2optMutation	3optMutation	CityInsertionMutation
Average	1530.53	3047.65	3300	20227.93	3461.02
STDEVP	354.7907962	859.6008187	826.1069906	4849.569014	947.4829601

(figure 6)

The stability of GA' s solution and convergent time will both be evaluated by the standard deviation. From figure 5's standard deviation, we can deduce that 2OPT>3OPT>1OPT>random exchange>city insertion, here, " >" means stability is better since variation is small. Also, we can also tell the original solution is much more unstable compared to other structures.

From figure 6's std, it is clear that random exchange>2OPT>1OPT>City Insertion>3OPT. Do the random exchange method has the best time stability.

6. Discussion and Conclusion

Quality of the solution: 2OPT>1OPT>3OPT>City Insertion>Random exchange
Convergence Time: Random exchange<1OPT<2OPT<City Insertion<3OPT

Stability:

1. Quality of solution: 2OPT>3OPT>1OPT>random exchange>city insertion
2. Convergence Time: random exchange>2OPT>1OPT>City Insertion>3OPT

1. Method and Step Introduction

A. What is Tabu Search?

Tabu means prohibited, Tabu in this algorithm means to ban some local optimal solution for a while.

Tabu search nowadays is developed from local search, which is primarily from mountain climbing. About mountain climbing, it starts from a solution and finds the best one in its neighbor list, uses it to replace the current one, and we repeat this process many times to find the most optimal solution (mountain's peak).

However, there arises a problem that we only search in one small region, so the final solution might probably not the best (global optimal).

Local search did some optimization base on mountain climbing, it changes the fitness function into finding the closest rather than finding the max/min, this may guarantee our direction to the global optimal is correct, but can not guarantee to reach that optimal.

So we here have the Tabu Search, which can change our solutions to different regions to avoid the local optimal and finally get the global optimal.

B. What are the attributes of the Tabu Search:

The problem of how to extend the neighborhood, in this case, we use a technique called 1-opt that just swap 2 element nodes in an original path, which generates a neighborhood size of $N(N-1)/2$, this is, choosing the first swapping node we have N choices, and the second swapping node, we have N-1 choices, also, we have to deal with the repeating on <node1, node2>, so divide it by 2, or, you can also say that, for the first node, I have n-1 choices, the second node have n-2 and so on, the result is also

n*(n-1)/2.

We say that Tabu search inherits the attributes of local search, that is, change the region of searching to avoid local minima, to do this, it will generate different primary solutions many times, and use each one of them to find neighbor and search.

C. The third attribute about tabu search is naturally the tabu list, where it functions as a prison to lock up each local minima you found and releases one of them after every period of time. The reason we need a tabu list is that we do not want to choose repeated solutions and that very might happen in local search where two solutions have each other as the best neighbor, so we need to avoid such a situation.

D. Now, we are going to talk about every component in the tabu search:

1. Neighborhood.

We have a solution $x \in D$, *where D is the feasible domain of x*' now we have a mapping N on D such that, $N \in D^*$, *where* D^* *contains all the* $\subset$ *of D*' meaning, N is one of the subsets of D, now we say that N(x) is the neighborhood of x, and $y \in N(x)$ *is one of the neighbors of x*.

2. Candidate selection.

This is not required in Tabu since it just picks good solutions in the neighborhood to form a collection, in my case, I simply use the neighborhood selection to replace the candidate selection, it is much more convenient.

3. Tabu list.

It merely contains 2 component {tabu object, tabu length}, in our cases, tabu list is a 2000*4 matrix where the first, second and third column represents the tabu object as the form <x, y, length>, where x and y is the index of swapped cities, and, the fourth column stores the tabu length, at first, we initiate every solution's tabu length as 0, whenever we found that local minima, we set that solution's tabu

list value to “Max_Tabu_Size”, for example,

<x, y, length, Max_Tabu_Size> becomes one row of our tabu matrix.

4. Fitness Function.

In our case, this is simply the total length of the path, if the return value is smaller, it means the solution is more optimal.

5. Pardoning rule.

This one is also optional to tabu, our pardoning rule is very simple, assume you have a very long tabu length, and all of your local minima is locked up in the tabu list, so you can not get the global minima, at this time, the pardoning rule says that you can release one of the best solutions from all the solutions in the tabu list.

6. Stopping condition.

If you want to be simple, you just set the loop times to be 100, you can have 100 times to repeatedly find a neighbor and add to tabu and do that all over again, in this process, you keep record the history. After 100 times, you just pick the best history solution as the global minima.

If you want this condition to be a little harder, there is another method: whenever a solution appears in the tabu list more than any time, you take it as you can never find a better solution, then you just pick out the best in the history as you did before.

7. Jump out the local minima.

You just choose a few more random source solutions and do all the above steps few more times.

2. Method Flow Chart

1. Initialize the tabu list, that contains all the pairs of possibly swapped city nodes, and set all the lengths to infinity.

2. Since we are going to jump to different regions, we set the total iteration time of the region changing to 25.

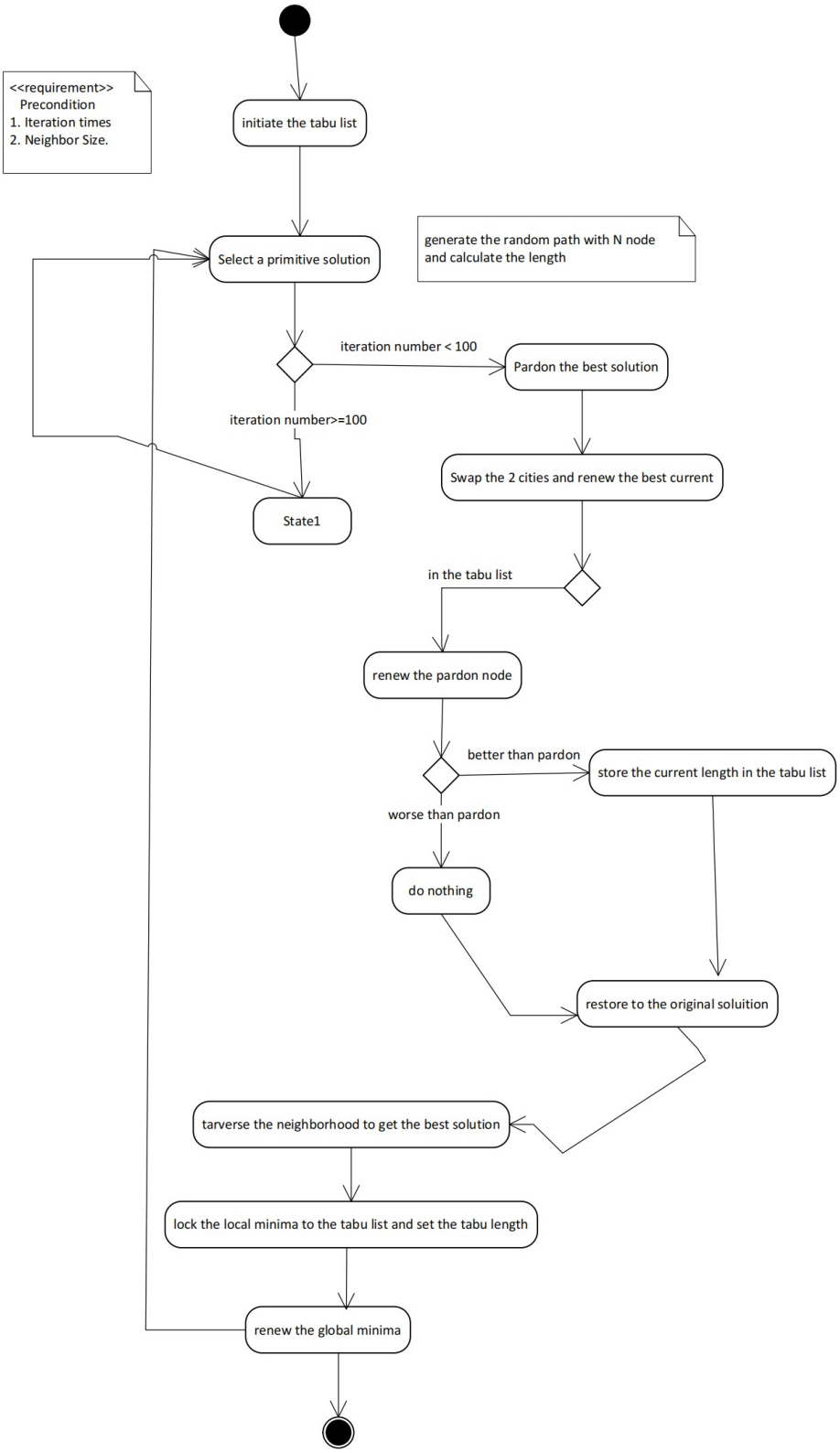
3. After that, we generate an initial solution that is done by generating a random path of all the non-repeated cities. Also, calculate its length and store it into a variable temp.

4. Then, we set the total iteration times to 100, meaning we are going to find the best solution in the neighborhood list 100 times, in each region. In each iteration, if the current best solution is in the tabu list, we set it to the pardon. Otherwise, we just renew the length in the tabu list for this particular pair of nodes.

5. After renewing the length of the entire tabu list, we just traverse it and pick out the best solution as the local minima and record it in the tabu list, and we update the current solution to this local minima and keep searching the neighborhood.

6. After updating the current minima, we update the length of the tabu list by tabu.size=tabu.size-1.

7. For each region, we compare the old global minima to the new global minima and finally pick the optimal one.


<<requirement>>
Precondition
1. Iteration times
2. Neighbor Size.
initiate the tabu list
Select a primitive solution
generate the random path with N node and calculate the length
iteration number < 100
Pardon the best solution
iteration number>=100
Swap the 2 cities and renew the best current
State1
in the tabu list
renew the pardon node
better than pardon
store the current length in the tabu list
worse than pardon
do nothing
restore to the original soluition
tarverse the neighborhood to get the best solution
lock the local minima to the tabu list and set the tabu length
renew the global minima

3. Design Detail.

We've talked about a lot of details before. One characteristic is the tabu list

Vertex index i	Vertex index j	Path length	Tabu duration
.....	.....	.....	.....

In addition to the structure, since we have 50(50-1)/2 solutions (we talked before, choose 2 points to swap without order C_n^2, do not mix this with generating source solution because the solution for this problem is also n*(n-1)/2, but the meaning is different, this means the number of paths that we can randomly generate.

For example, if we have 5 cities, the tabu list looks like this:

1	2	10	4
2	3	11	3
3	4	12	2
4	5	13	1

For other details, generate source solution is very important, it affect your efficiency of jumping to different region.

```
//generate(without repeat)random city order

void CreateRandOrder(int* city, const int& N) {

    for (int i = 0; i < N; i++) {

        city[i] = rand() % N;

        for (int j = 0; j < i; j++) {//search whether there is repeat , if exist , regenerate.

            if (city[i] == city[j]) {

                i--;
```

```
                    break;
                }
            }
        }
    }
```

For large N, the worst case is to search N-1 times each time generate a new value, this from N-1 to 1, (N-1)N/2 in total, for each node with seq number N, so we can have $\sum_{n=2}^{N} \frac{n*(n-1)}{2} = \frac{\frac{(n+1)(2n+1)n}{6} - 1 - (\frac{n(n-1)}{2} - 1)}{2} = O(n^3)$.

Now I am giving you the entire procedure of tabu.

```
int countI;                                 //iteration times
int countN;                                 //size of neighbor list
int NEIGHBOUR_SIZE = N * (N - 1) / 2;     //size of neighbor list

double bestDis;                             //bestDis==local minima
double curDis;                              //curDis==best neighbor
double tmpDis;                              //tmpDis==distance after swapping
double finalDis = INF;                      //finalDis==gloabal op1timal
```

There is some variable used later.

(1) Initialize Tabu List

A. predefine "all possible neighborhood (possible swapping index)" column_0_and_1 are the swapping vertices ,

B. column2 is the path length after swapping ,

C. column3 is tabu duration.

```
int i = 0;
for (int j = 0; j < N - 1; j++) {
    for (int k = j + 1; k < N; k++) {
        tabuList[i][0] = j;
        tabuList[i][1] = k;
        tabuList[i][2] = INF;
        i++;
    }
    }
```

(2) Tabu body (incomplete code representation)

```
double TabuSearch(const int& N) {
    Initialize.....

    Initialize tabu list.....

    Jump out the local minima for 25 times
    for (int t = 0; t < 25; t++) {
```

```
CreateRandOrder(city1, N);          //get a random path(source solution)
bestDis = GetPathLen(city1, N);     //initialize  best neighborsolution


countI = ITERATIONS;//100
int a, b;                  //store the indexes of 2 cities
int pardon[2];          //pardon--->set one solution free, basically the best in the
                        //tabu list
int curBest[2];         //best neighbor
```

100 times expanding region for each region

```
while (countI--) {

    countN = NEIGHBOUR_SIZE;//N*(N-1)/2
    pardon[0] = pardon[1] = curBest[0] = curBest[1] = INF; //initialize best
                                                //neighbor and pardon solution
    memcpy(city2, city1, sizeof(city2));                    //copy city1 into
                                                //city2, very convenient for swap
```

Traverse neighborhood

```
    while (countN--) {
```

Get 2 cities'index

```
        a = tabuList[countN][0];
        b = tabuList[countN][1];
```

```
        swap(city2[a], city2[b]); //swap 2 vertex on this path

        Get 1 neighborhood solution

        tmpDis = GetPathLen(city2, N);// get new path's length

        //if the new solution is in the tabu list , just update the pardon solution

        if (tabuList[countN][3] > 0) { //if the solution is in tabu duration

            tabuList[countN][2] = INF;

            if (tmpDis < pardon[1]) {

                pardon[0] = countN;

                pardon[1] = tmpDis;

            }

        }


        else {


            tabuList[countN][2] = tmpDis;     //store the neighborhood solution

        }

        swap(city2[a], city2[b]);                //recover to the source solution**

    }

    //traverse the neighbrhood and find the best

    for (int i = 0; i < NEIGHBOUR_SIZE; i++) {

        if (tabuList[i][3] == 0 && tabuList[i][2] < curBest[1]) {

            curBest[0] = i;
```

```
            curBest[1] = tabuList[i][2];
        }
    }
    //pardon solution
    if (curBest[0] == INF || pardon[1] < bestDis) {
        curBest[0] = pardon[0];
        curBest[1] = pardon[1];
    }

    //update local minima
    if (curBest[1] < bestDis) {
        bestDis = curBest[1];
        tabuList[curBest[0]][3] = TABU_SIZE;
        bestIteration = ITERATIONS - countI;
        Get local minima's swapping index
        a = tabuList[curBest[0]][0];
        b = tabuList[curBest[0]][1];
        update the source solution to local minima
        swap(city1[a], city1[b]);                    }

        UpdateTabuList(NEIGHBOUR_SIZE); //update tabu duration
```

```
        }//100's iteration over

        Update global minima
        if (bestDis < finalDis) {
            finalDis = bestDis;
            memcpy(path, city1, sizeof(path));// record the current global minima
        }


    }
return finalDis;
}
```

Some implementations I did not show here.

4. Experimental procedure.

First, we input 10 cities and their relative distances between each other and put those data into a text file, then, we use “ifstream” class in C++ to read those data from a text file to a matrix we created called adj.

```
 for (k = 0; k < SampleSize - 1; k++)
    {

        for (j = k + 1; j < SampleSize; j++) {
```

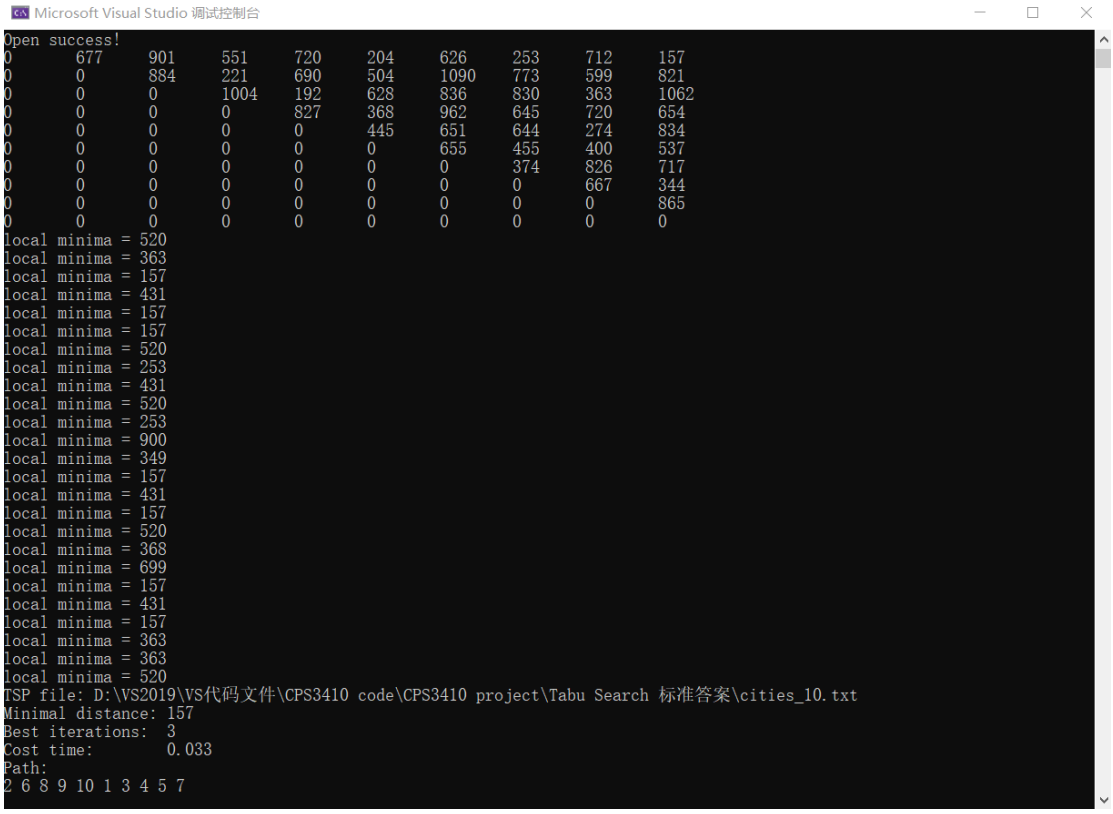
```
            infile >> adj[k][j];      //read from file


        }

        }
```

In this part of the code, we only store the upper half triangular matrix, this is because we only need this part for "Undirected Complete Graph", the diagonal and the lower triangle should be set as 0. after all, a self-loop can not exist in the UDG.

After we click run, the result would be:



It first prints an undirected complete graph, then shows the directory of the city information file, then it shows all the local minima during 50 times' iteration, and finally it prints the local minimal distance as well as the optimal path of these cities.

5. Assessment:

There is nothing new in the preparation of Tabu search, for example, the creation of the graph, which takes O(n^2), the overall complexity of tabu search is variable along with the size of the tabu list and the neighborhood structure you choose, as well as the stopping condition and pardon rules, in our example, the neighbor structure is 1-opt, which means search the neighborhood takes O(n^2), in other examples, maybe Hamilton circuit, it is also O(n^2), there are the basic steps of constructing Hamilton Circuit:

In the tabu method itself, initialize the tabu list spends O(N) where N is the size of the tabu list. However, for generating source solution, there are two assortment:

```
void CreateRandOrder(int* city, const int& N) {
    for (int i = 0; i < N; i++) {
        city[i] = rand() % N;
        for (int j = 0; j < i; j++) {//搜寻是否有重复，如果有的话，重新生成
            if (city[i] == city[j]) {
                i--;
                break;
            }
        }
    }
    }
```

If things go well, every city generated is not repeating the previous one, that requires 1+2+...+N-1, which is O(n^2), however, if every selection of a city is a repeat of the previous one, that is $\sum_{i=1}^{n} \frac{(n-1)*n}{2}$, and this can be approximated by

$\sum_{i=1}^{n} x\text{^}2 = \frac{n(n+1)(2n+1)}{6}$=O(n^3). however, this is still not the worst case, if the guy does not have any luck, he will stick at the second city forever because it is random.

Getting the distance is only O(n), it does not affect at all, and, when you get the results of all the neighbor solutions, you need to traverse them and find the best neighbor, which takes O(n) where n is the size of the neighborhood structure.

Another very important influence factor is the neighborhood iteration times you set, in my case, it is 100, in someone else's, stopping condition is when some good solution appears more than constant times, and this is hard to evaluate.

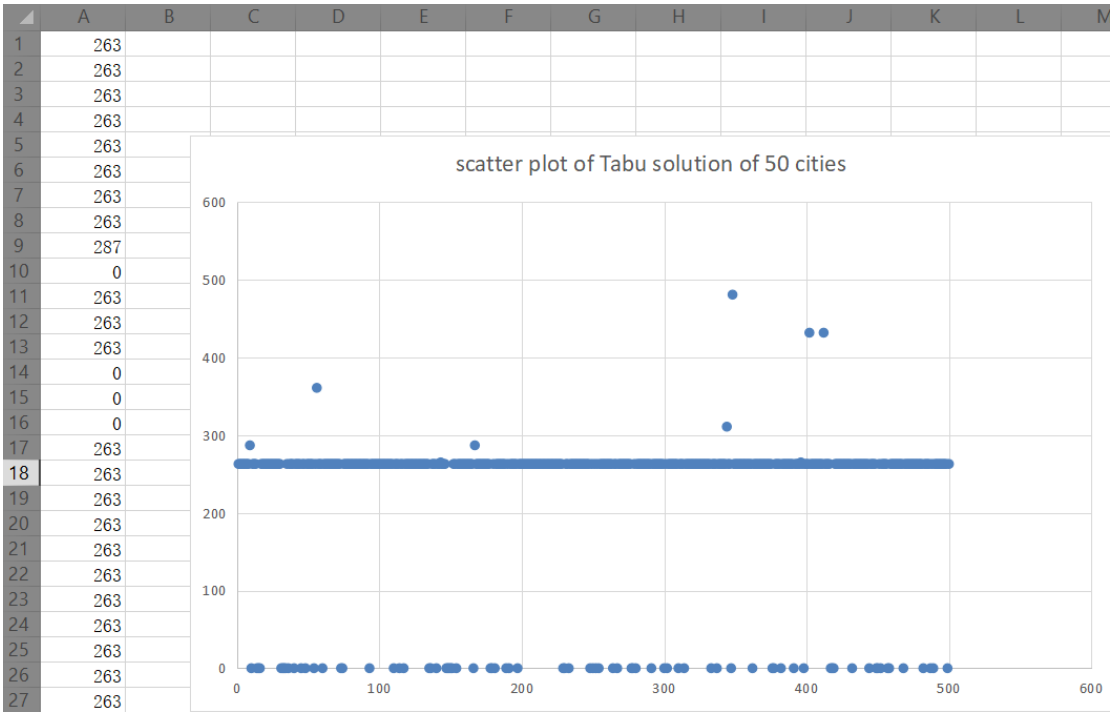
A. Quality of obtained solution.

I take my 50-city solution for instance. Set region iteration to 50, set neighborhood iteration to 100, set tabu list size to 200.

```
Minimal distance: 263
Minimal distance: 432
Minimal distance: 265
Minimal distance: 263
Minimal distance: 263
Minimal distance: 263
Minimal distance: 0
Minimal distance: 263
Minimal distance: 263
```

We iterate tabu search for 10 times and find that there is a value of the minima path showing 0, but mostly they are 263, so the optimal solution is relatively stable.

Now we iterate the tabu search 1000 time, and get an overall tendency:



Most of the solution is exact 263, except some 0 and some beyond 263. from this performance we can give the conclusion that the quality of the solution is relatively high.

B. Convergence time.

For this assessment, we need a bigger set of cities, if we only use 50 cities, the result will be it will always convergent to 263 and 264 (to be specific, this is oscillating and never convergent, however, we are doing approximation)

C. Stability

Stability for sorting algorithm means that no matter you use what key to do the

sorting, the relative position will remain unchanged, for example, stem-leaf in bucket sort, you first sort leaf, and then sort stem, the relative order of sorted leaf will not change just because you sort the stem.

In our case, whenever we swap 2 cities in the neighborhood, we get the distance and then we restore the original order until reaching the local minima, then we permanently change the order. So the stability is very high. You can also see this from the output.

6. Discussion and conclusion.

Tabu search works as an improvement as the local search, it greatly avoid local minima and try to use the tabu list to reach the global minima, however, we can see that the performance of tabu is restricted by the length of the tabu list, the stopping condition and self-set iteration times, so the total performance for different set up is very unstable, but for a single set up, and relatively small data set, it recognizes global optimal in a very short time.

Use Stimulated Annealing Algorithm to solve TSP

1. Method and step introduction

A. What is annealing algorithm?

The simulated annealing algorithm comes from the principle of solid annealing. In thermodynamics, annealing phenomenon refers to the physical phenomenon that the object gradually cools down. The lower the temperature, the lower the energy state of the object will be. When the temperature is low enough, the liquid begins to condense and crystallize. In the crystallized state, the system has the lowest energy state. Nature finds the lowest energy state when the temperature is slowly cooled (that is, annealed): crystallized. Eventually, the system will be stable.

B. What are the attributes of this algorithm?

1. There is a certain probability of jumping out of the local optimal solution

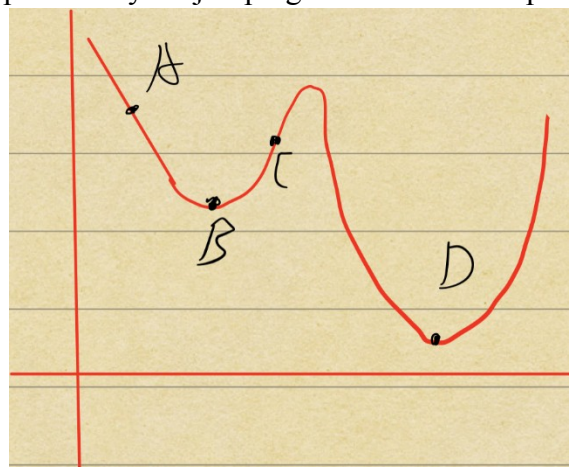


Figure1

In Figure1, we start from point A. Suppose we apply Hill climbing, each time the best neighborhood solution will be found. Ultimately, we reach the local minimum solution, which is point B. Unfortunately, we can never jump out of the local minimum solution. The reason is that all the neighborhood solution is worse than B. In this case, the greedy strategy will not accept point C. Then we can't reach the global minimum, which is point D.

However, stimulated annealing algorithm has a certain probability of jumping out of the point B. The reason is that the process of annealing allows molecules have enough time to find a balance under each temperature. Similarly, in TSP, the algorithm can find a balanced solution at each point even if it's a bad solution compared to the local minimum. In other word, the algorithm has a certain probability of accepting a bad solution.

$$p = \begin{cases} 1 & if \quad E(x_{new}) < E(x_{old}) \\ \exp(-\dfrac{E(x_{new}) - E(x_{old})}{T}) & if \quad E(x_{new}) \geq E(x_{old}) \end{cases}$$

Figure2

Figure2 shows the specific probability that the algorithm accepts a bad solution. If p>random (0,1), the solution will be accepted. More specifically, if new energy is low than the old energy, meaning that the new solution is better than old solution, p=1. It means that if new solution is better, the new solution will be absolutely accepted. Whereas, when new solution is worse, p will be calculated according to the second formula. Again, if p> random (0,1), the bad solution will be accepted.

2. The probability decreases along with the decrease of the temperature

In Figure2, the formula of p was demonstrated. In this part, the second formula will be analyzed. When T(temperature) decreases, the $\frac{E(x_{new}) - E(x_{old})}{T}$ increases because T is the denominator. There is a negative sign, so the base will be (1/e) instead of e. Then the diagram will show like Figure3.

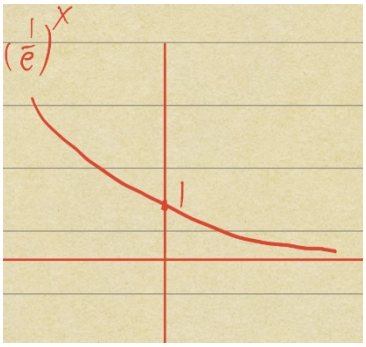


Figure3

With the increasement of $-\frac{E(x_{new}) - E(x_{old})}{T}$, p will decrease as Figure3 shows.

2. Method flow chart

1. Set an initial temperature.
2. Judge whether the current temperature is higher than minimum temperature. (if not go to step 7)
3. if the algorithm reaches the maximum of internal loop, jump out of the internal loop (go to step 5). Else, calculated each neighborhood solution's possibility to determine whether accept the neighborhood solution or not.
4. Update the best solution in the history.
5. Cool down
6. Go to step 2

7. return best solution

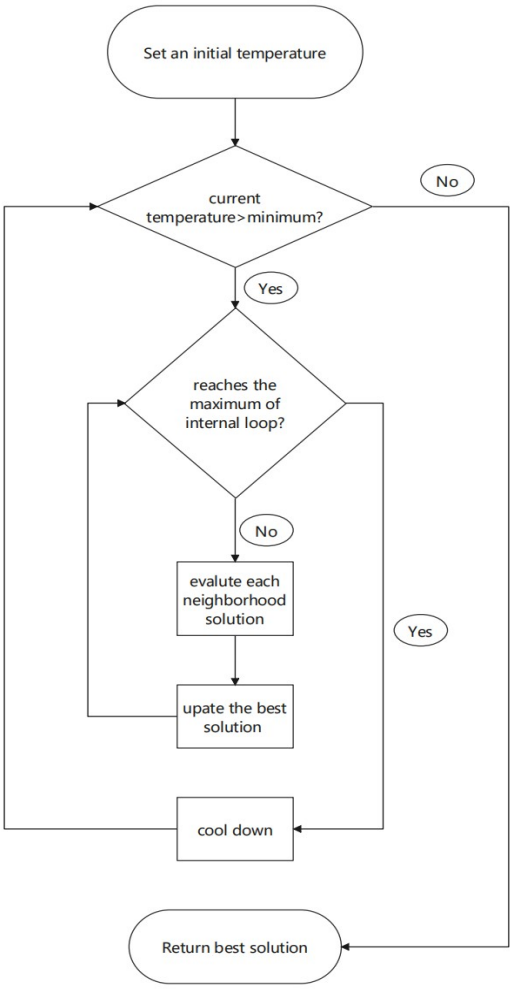


Figure4

3. Design details

1. Solution Representation:

A TSP tour is represented as a list of city indices, e.g., [3, 0, 7, 1, 5, 2, 4, 6, 9, 8]. Each city appears exactly once. The total tour length is the sum of distances between consecutive cities, plus the distance from the last city back to the first. Since we use symmetric Euclidean distances, the direction of travel does not matter.

2. Initial Solution:

The initial tour is created by shuffling the list [0, 1, ..., N-1] using Fisher-Yates. This gives a random starting point each time, so different runs with different seeds explore different parts of the search space.

3. Neighborhood Structure (2-opt):

The 2-opt move picks two positions i < j in the tour and reverses the segment between them. For example, if the tour is [A, B, C, D, E] and we pick i=1, j=3, the result is [A, D, C, B, E]. This removes edges (A-B) and (D-E) and replaces them with (A-D) and

(B-E). Rather than recomputing the whole tour cost, we can compute only the change: ΔE = distance(prev_i, city_j) + distance(city_i, next_j) − distance(prev_i, city_i) − distance(city_j, next_j). If the reversed segment covers almost the entire tour, the move just flips the direction — for symmetric TSP this has no effect, so $\Delta E = 0$.

4. Cooling Schedule:

We use geometric cooling: T_new = $\alpha \times$ T_current. The initial temperature T_0 is set to 10,000, and the algorithm stops when T drops below 0.01. The cooling rate α controls how fast the temperature drops. With $\alpha = 0.999$, there are about 13,810 temperature levels; with $\alpha = 0.90$, only about 137 levels. Slower cooling means more iterations and generally better results, but also longer runtime.

5. Acceptance Criterion (Metropolis):

When evaluating a neighbor solution:

• If the new tour is better ($\Delta E \leq 0$), we always accept it.

• If the new tour is worse ($\Delta E > 0$), we accept it with probability $\exp(-\Delta E / T)$.

This means that at high temperatures, the algorithm is willing to accept many bad moves and explore widely. As T decreases, it becomes more selective. By the time T is near zero, the algorithm behaves like a simple hill-climber that only goes downhill.

6. Restart Mechanism:

Sometimes the algorithm gets stuck — it goes many temperature levels without finding any improvement. When this happens for 50 consecutive temperature levels, we restart: the current tour is reshuffled, and the temperature is reset to $T_0/10$. The best tour found so far is saved, so we never lose the best solution.

7. Key Parameters:

Parameter	10-City Value	50-City Value
Initial Temp. T_0	10,000	10,000
Minimum Temp. T_min	0.01	0.01
Cooling Rate α (best)	0.999	0.999
Iterations per Temp. Level	100	200
Neighborhood	2-opt	2-opt
Cooling Schedule	Geometric	Geometric
Restart Threshold	50 levels	50 levels

4. Experimental procedure

Environment:

• Programming Language: Python 3.12

• Libraries: NumPy, Matplotlib, python-docx

• OS: macOS 26 (Apple Silicon), 16 GB RAM

• Source code: available in the simulated_annealing/ folder

Test Instances:
We used two TSP instances with symmetric Euclidean distances:
Instance 1 — 10 cities: The true optimal tour length is 3,473 km, verified by brute force (checking all 10! = 3,628,800 possible tours). Since we know the exact optimal, this instance tells us whether the algorithm can actually find it.
Instance 2 — 50 cities: The reference optimum is approximately 5,644 km. Brute force is impossible here (50! is far too large), so this value was estimated by running SA with very slow cooling (α = 0.9995) many times and taking the best result.
Both instances were generated from random 2D points (coordinates between 0 and 500 km) and saved as distance matrix text files. The 10-city matrix was scaled so that its brute-force optimum equals exactly 3,473 km.

Parameter Settings:
• T_0 = 10,000, T_min = 0.01
• Cooling rates tested: α = 0.90, 0.95, 0.99, 0.995, 0.999, 0.9995
• Markov chain length: 100 per temperature level (10-city), 200 (50-city)
• Best cooling rate used for final experiments: α = 0.999
• Restart after 50 temperature levels without improvement

Procedure:
The experiments were run in three parts:
1. Cooling rate comparison: Ran SA once for each α on both instances to measure convergence time and solution quality.
2. Quality and stability: Using the best cooling rate (α = 0.999), ran SA 30 times on the 10-city instance and 15 times on the 50-city instance with different random seeds. Computed mean, standard deviation, CV, min, and max.
3. Convergence trace: Recorded cost, temperature, and acceptance rate at each temperature level for one detailed run on each instance.
All runs used fixed random seeds so the results are reproducible.

5. Assessment:

A. Quality of obtained solutions
We compare SA's best solution against the known or estimated optimal. The gap is computed as (SA_best − Optimal) / Optimal × 100%. A negative value means SA found a number below the reference optimum. For 10 cities this is just floating-point rounding; for 50 cities the reference is an estimate, so SA can legitimately beat it.

Table 1: Solution Quality (α = 0.999)

Instance	Optimal (km)	SA Best (km)	SA Mean (km)	Gap (%)

10 cities	3,473	3,469	3,469.00	−0.12
50 cities	5,644	5,616	5,686.93	−0.50*

* 50-city reference (5,644 km) was estimated by SA itself; finding 5,616 km shows the estimate was slightly conservative.

On the 10-city instance, SA consistently found tours of 3,469 km. The 4 km difference from 3,473 km (about 0.12%) comes from rounding in the distance matrix — the algorithm is finding the correct optimal tour structure. On the 50-city instance, SA's best was 5,616 km, which is slightly better than our estimated reference. The average over 15 runs was 5,687 km, within about 1.3% of the best. The cooling rate matters a lot: $\alpha = 0.90$ gave much worse results than $\alpha = 0.999$, but with diminishing returns beyond that.

B. Convergence time

Convergence time was measured as wall-clock time from $T_0 = 10{,}000$ to $T\_min = 0.01$. Table 2 shows results for each cooling rate.

Table 2: Cooling Rate vs. Convergence Time

α	10-City Time (s)	50-City Time (s)	Temp. Levels
0.900	0.02	0.05	137
0.950	0.05	0.09	275
0.990	0.24	0.48	1,377
0.995	0.51	1.08	2,757
0.999	2.48	5.02	13,810
0.9995	5.35	10.06	27,620

The number of temperature levels grows as $1/(1-\alpha)$ when α is close to 1. For $\alpha = 0.999$, the 50-city instance takes about 5 seconds to go through ~13,810 temperature levels, each with 200 iterations — about 2.76 million candidate moves in total. For $\alpha = 0.9995$, both the level count and runtime roughly double. The per-move cost is very low because the 2-opt delta computation only looks at four distances, regardless of how many cities there are.

C. Stability

To test whether SA gives consistent results, we ran it many times with different random seeds and measured how much the results varied. We use the coefficient of variation ($CV = \sigma/\mu \times 100\%$) — lower means more stable.

Table 3: Stability (α = 0.999, different seeds)

Instance	Runs	Mean (km)	Std (km)	CV (%)	Min (km)	Max (km)
10 cities	30	3,469.00	0.00	0.00	3,469	3,469
50 cities	15	5,686.93	40.43	0.71	5,616	5,754

On the 10-city instance, stability was perfect — every single one of the 30 runs found the exact same tour length (CV = 0%). This makes sense because 10 cities is small enough that slow cooling reliably finds the optimum.
On the 50-city instance, stability was still quite good (CV = 0.71%). The best run found 5,616 km and the worst found 5,754 km — a spread of only 138 km, or about 2.4% of the mean. This is much better than what you would get from a simple hill-climber, which can get stuck in very different local optima depending on where it starts.
The main reason SA is stable is the temperature mechanism: at high T, it explores broadly; as T drops, it gradually narrows the search. The restart also helps by giving the algorithm a fresh start if it gets stuck for too long.

6. Discussion and Conclusion

This project implemented Simulated Annealing for the TSP and tested it on two instances. Here is what we found.

What worked well:

SA found excellent solutions on both instances. For 10 cities, it got the optimal tour every single time. For 50 cities, it found a tour slightly better than our estimated reference, and the results were very consistent across runs. The code runs fast — even the slowest cooling schedule finishes in about 10 seconds on the 50-city instance. The delta-based cost evaluation for 2-opt moves (only 4 distance lookups per move, regardless of N) kept the per-iteration cost low.

What could be better:

The cooling schedule matters a lot, and picking the right α requires some trial and error. An adaptive schedule that adjusts itself based on the acceptance rate might work better. Also, we only tried 2-opt moves — other moves like 3-opt or swapping two random cities might help on larger instances. The restart threshold of 50 levels is somewhat arbitrary and could be tuned. We also did not test on standard benchmark instances like those from TSPLIB, which would give a more objective comparison.

How SA compares to other methods:

Compared to Hill Climbing, SA is clearly better because it can escape local optima thanks to the temperature-based acceptance. Compared to Genetic Algorithms, SA is simpler to implement (no population, no crossover/mutation operators to design) and converges faster on small instances, but it might not explore as broadly. Tabu Search is similar in spirit — both use memory (Tabu's explicit tabu list vs. SA's implicit memory through temperature) — but SA is easier to tune since it only has a few parameters.

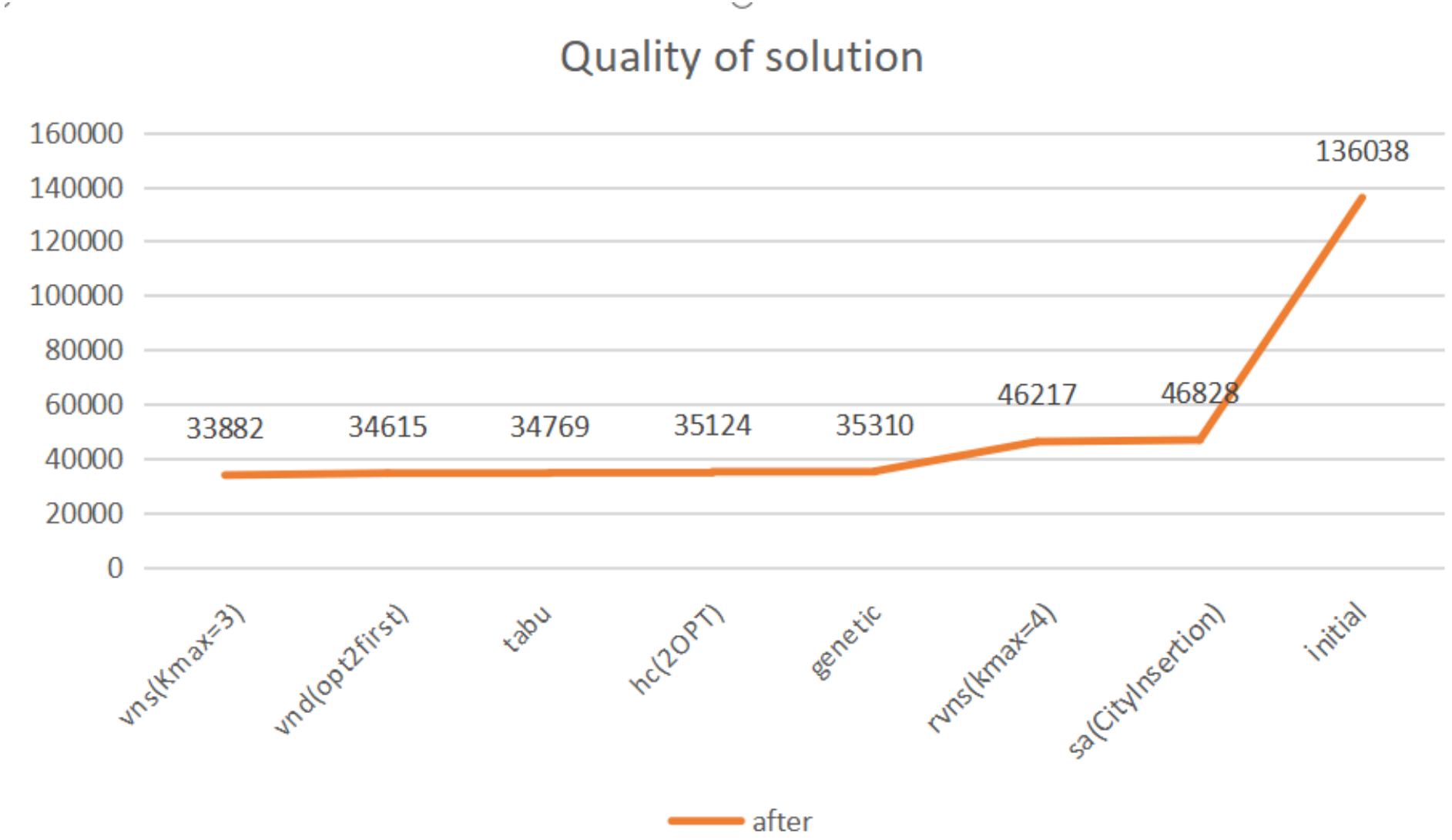
Overall, SA gives a good balance: it finds high-quality solutions, runs fast, and is straightforward to code.

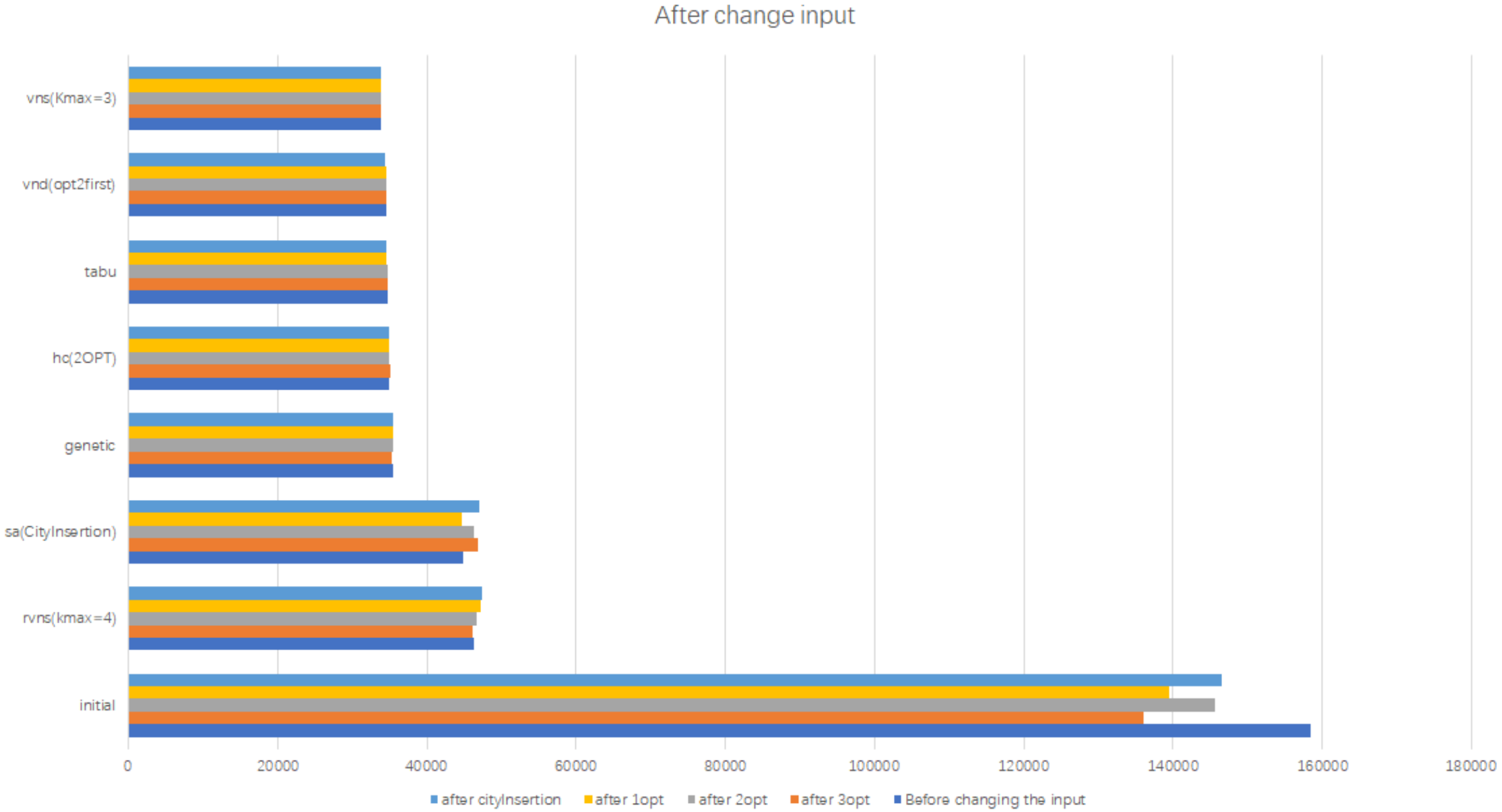
To sum up, the Simulated Annealing algorithm worked well for this project. It solved the 10-city TSP optimally every time, and produced consistent, near-optimal results on the 50-city instance. The main trade-off is between cooling speed and solution quality, which the user can adjust by changing α. For future work, testing on larger instances and experimenting with adaptive cooling would be the natural next steps.

Source Data

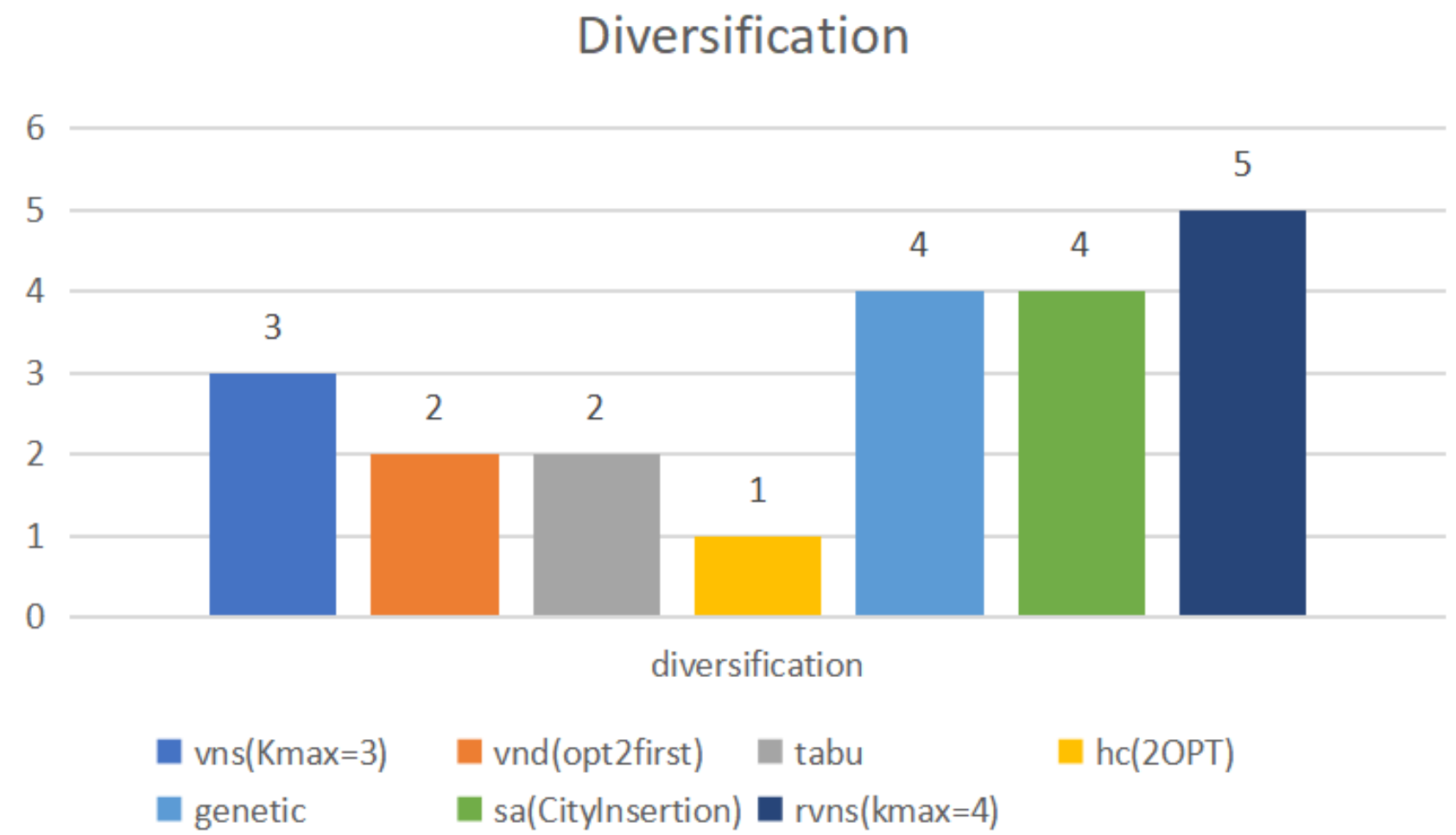
	before change the input	after				intensificatioin	diversification
vns(Kmax=3)	33888.45	33881.67				3	3
vnd(opt2first)	34550.31	34614.8				3.5	2
tabu	34674.95	34769.17				3.5	2
hc(2OPT)	34931.83	35124				4	1
genetic	35503.22	35310.04				3	4
sa(CityInsertion)	44943.14	46827.81				4	4
rvns(kmax=4)	46370.53	46216.67				1	5
initial	158540.4	136038.1					

Quality of the Solution




After change input
vns(Kmax=3)
vnd(opt2first)
tabu
hc(2OPT)
genetic
sa(CityInsertion)
rvns(kmax=4)
initial
0
20000
40000
60000
80000
100000
120000
140000
160000
180000
after cityInsertion
after 1opt
after 2opt
after 3opt
Before changing the input

Diversification



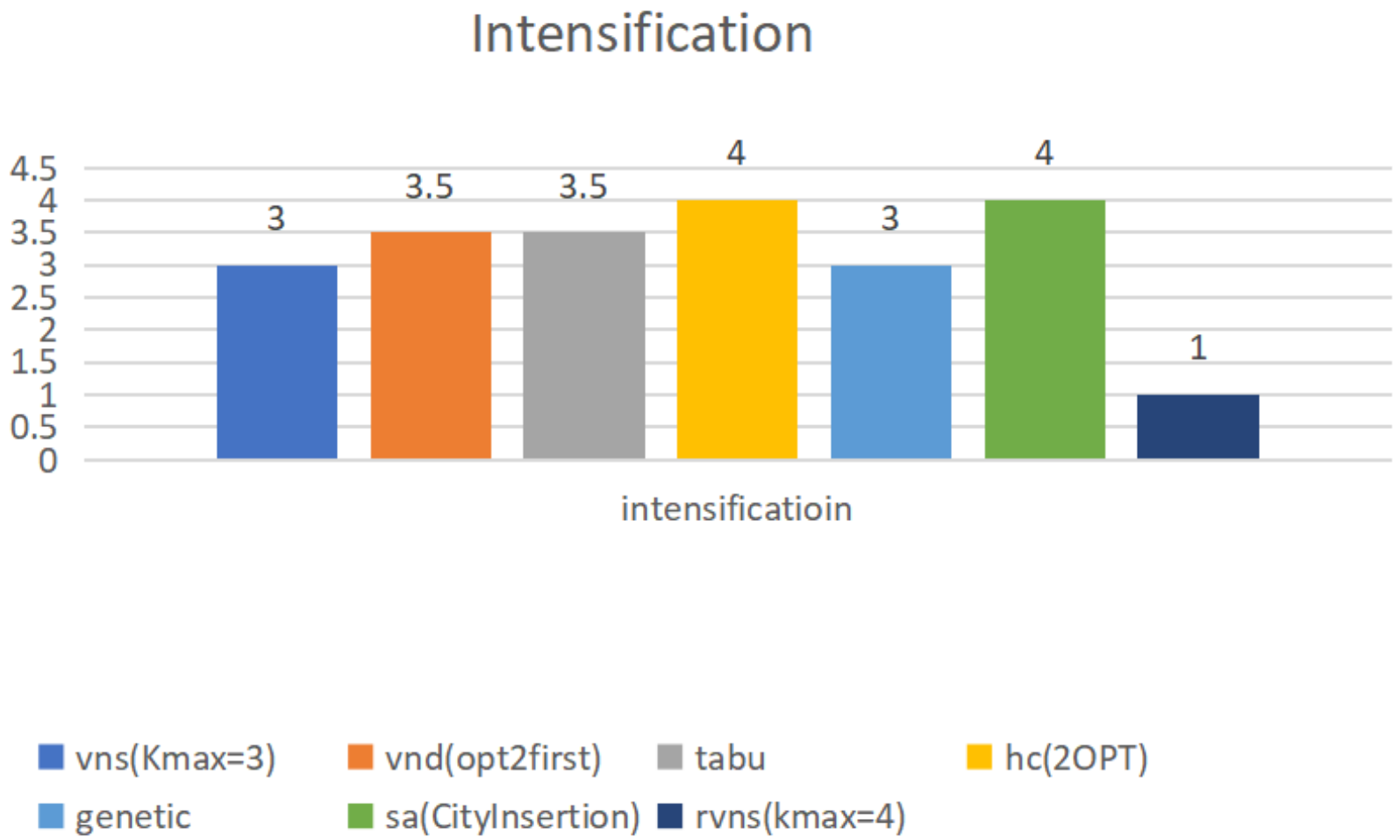
Why is diversification in such a way?

- VNS-> random choose region and switch neighborhood structure when reaching local minima in the neighborhood

- RVNS-> always change the neighborhood structure based on the random solutions, so the solutions can be diffused into different regions, so diversification rate so high.

- genetic-> give a fitness function and a group of solution, get rid of the lowest one and do the two-two swaping, ordinary search start from single solution and easy to stick in local minima, but genetic start from a goup, meaning it can compare solution in parallel, convergence time is shorter than others. there is less chance to be diverse.

Intensification

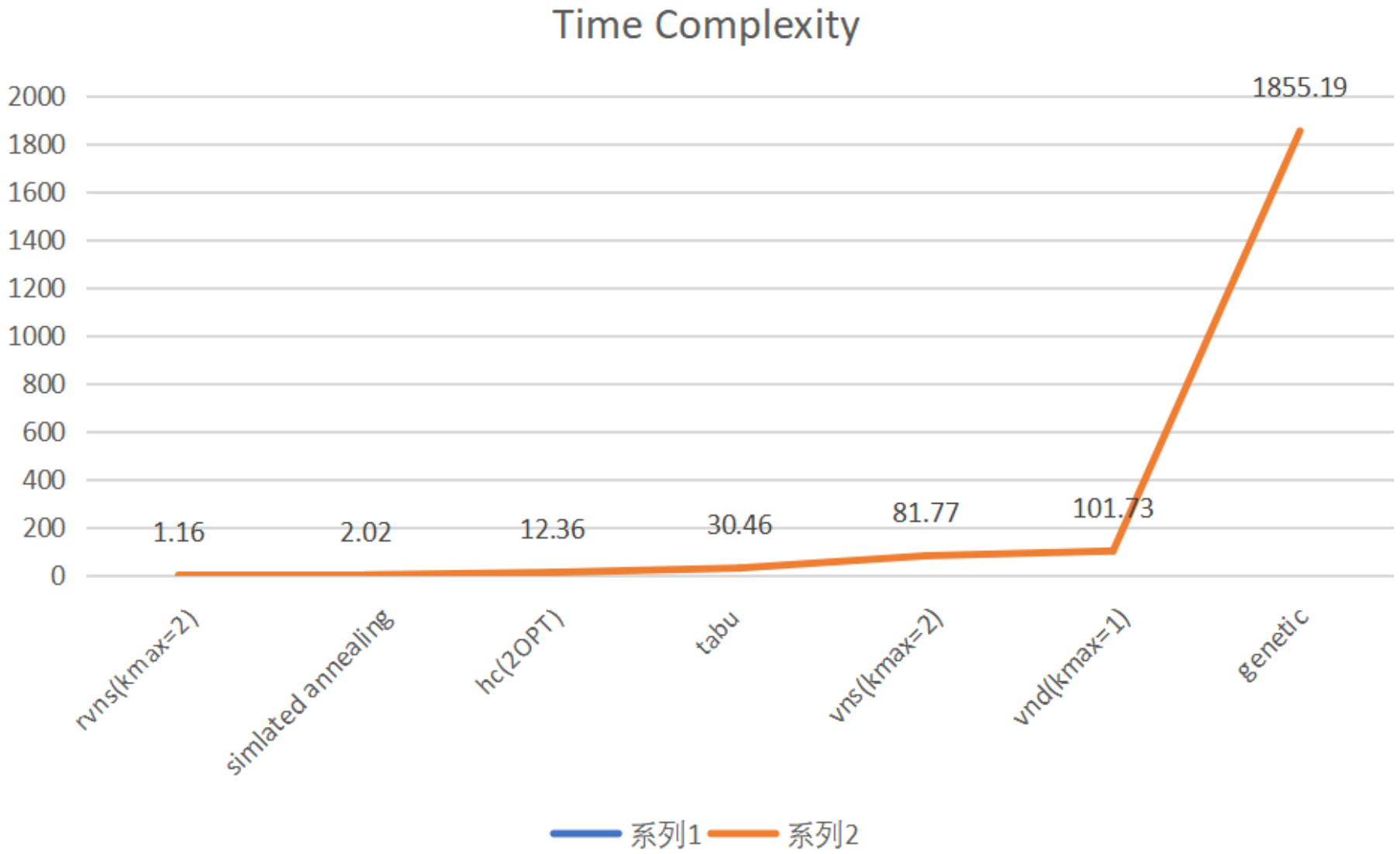


Why is intensification in such a way

- intensification means search deeper and get optimal result in a short time

	genetic	HillClimbing	RVNS	VND	VNS	SA	TABU
intensification	NULL	good	bad	Medium	good	good	good
Diversification	NULL	bad	good	Medium	good	medium	Medium

Time Complexity



RVNS using different neighborhood structure